

You will also need to install the Node package manager (*npm*) for development, which can be done using the following command:

```
sudo apt install npm
```

Once the installation is complete, verify it using the following command:

```
npm -v
```

The latest Docker version may be missing in the Linux repository; so you can download it from Docker's official repository. First update the existing list of packages using the code below:

```
sudo apt update
```

In order to install a few other packages to allow the use of *apt* over HTTPS, use the following command:

```
sudo apt install apt-transport-https ca-certificates curl software-properties-common
```

Then you will need to add the GPG key for the official Docker Repository on your system, as follows:

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
```

Now add the Docker Repository to APT sources:

```
sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu bionic stable"
```

Next, update the package database with Docker packages from the newly added repo, as follows:

```
sudo apt update
```

Finally, install Docker using the command given below:

```
sudo apt install docker-ce
```

After Docker is installed, its daemon will be started, and the process will be enabled to start on boot. This can be checked using the following command:

```
sudo systemctl status docker
```

By default, Docker can only be run by the root user or by a user in the Docker group, which is automatically created during Docker's installation.

Creating a Node.js application

Now that you have already set up your development environment, you will need an application for containerisation. Let us develop a basic application using Node.js which we will Dockerise later in this article.

As you already have *npm* installed, let us start with installing Express and Express-generator globally, using the following commands:

```
npm i -g express express-generator
```

In this step, we will create a skeleton of an Express application named *node-docker*. Use the command given below to create a skeleton:

```
express node-docker
```

The app will be created in a folder named *node-docker*. Then install dependencies by changing the directory to *node-docker* and use the following command:

```
npm install or npm I
```

The application can be started using the *npm start* command. Let us use this same sample application for Dockerisation.

Writing a Dockerfile

Dockerfile is a text file that contains all the commands that are called in order to assemble an image. We

will need this file to be added to the directory of our application that is in the *node-docker* folder. For our application, Dockerfile will instruct Docker to download Node.js, create a directory for the application, and also install all the packages required by the application.

```
FROM node:10
#Create app directory
WORKDIR /usr/src/app
#Install app dependencies
COPY package*.json ./
RUN npm install
# Bundle app source
COPY . .
EXPOSE 8080
CMD [ "npm", "start"]
```

A valid Dockerfile must start with a *FROM* instruction, which initialises a new build stage and sets the base image for subsequent instructions. It can appear many times within a single Dockerfile in order to create multiple images or use every build stage as a dependency for another.

The *WORKDIR* instruction sets the working directory for all the other following instructions.

The *COPY* instruction copies the files from the application directory to a container's file system.

The *EXPOSE* instruction informs Docker that the container will listen on a specified port at runtime. This instruction does not actually publish the port but is used as documentation for building the image and for running the container. To actually publish, the port *-p* is used at the time of running the container with the *docker run* command.

The *CMD* instruction provides default commands for an executing container. When used in the shell or exec format, it sets the command to be executed when running the image. If a Dockerfile has more than one *CMD* instruction, only then will the last instruction be executed.